



CYT_LAB ASSIGNMENT

K V S CHANAKYA VAMSI | 261100690013 | 19.08.2026

Capturing and Analyzing Login Credentials
over HTTP and HTTPS Using Wireshark

OBJECTIVE

To examine how login credentials are transmitted over HTTP and HTTPS, Wireshark is used on a separate attacker machine to capture the network traffic. The captured data is then compared to determine what a network sniffer can read in each case. The portal, username, and password remain identical for both tests, with only the transport protocol being changed. This allows the impact of encryption on credential security during transmission to be clearly observed.

LAB SETUP

- Docker Container Startup

The Docker Compose command builds the lab image and starts the login portal container.

This establishes the environment required for testing HTTP and HTTPS traffic.

```
(kali@kali)~/wireshark_login_lab
$ docker compose up -d --build
[+] Building 2.4s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 1.05kB
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load metadata for docker.io/library/python:3.11-slim
=> [auth] library/python:pull token for registry-1.docker.io
=> [internal] load build context
=> => transferring context: 8.58kB
=> [1/4] FROM docker.io/library/python:3.11-slim@sha256:5f3c7c4a...
=> [2/4] WORKDIR /app
=> [3/4] COPY requirements.txt /app/
=> [4/4] RUN pip install --no-cache-dir -r requirements.txt
=> exporting to image
=> => exporting layers
=> => writing image sha256:6a1b2f9d7c4c6b9ff2d8e9a1b5c3e4d6f7a8b9c0
=> => naming to docker.io/library/wireshark_login_lab
[+] Running 2/2
✓ Network wireshark_login_lab_default Created
✓ Container wireshark_login_lab-1 Started
```

Fig – I: *docker compose up -d --build* builds image and starts up wireshark container on the host

The *docker compose ps* command confirms that the container is running and that both HTTP (port 80) and HTTPS (port 443) are available for the experiment.

```
(kali@kali)~/wireshark_login_lab
$ docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
wireshark_login_lab-1	wireshark_login_lab	"python app.py"	web	15 seconds ago	Up 14 seconds	0.0.0.0:80->80/tcp, 0.0.0.0:443->443/tcp

```
Network wireshark_login_lab_default Created
```

Fig–II: Running Docker container with HTTP and HTTPS ports published

- Capturing the Traffic – Sending HTTP Login

The target machine is first verified to be reachable using a ping. The login credentials are then submitted to the portal using HTTP, sending the username and password without transport encryption.

```
(kali@kali)~$ ping -c 4 192.168.56.101
PING 192.168.56.101 (192.168.56.101) 56(84) bytes of data:
64 bytes from 192.168.56.101: icmp_seq=1 ttl=64 time=0.493 ms
64 bytes from 192.168.56.101: icmp_seq=2 ttl=64 time=0.437 ms
64 bytes from 192.168.56.101: icmp_seq=3 ttl=64 time=0.448 ms
64 bytes from 192.168.56.101: icmp_seq=4 ttl=64 time=0.461 ms

---
(kali@kali)~$ curl -v -X POST http://192.168.56.101/login \
> -H "Content-Type: application/x-www-form-urlencoded" \
> --data "username=student&password=LabPass@2026"
Note: Unnecessary use of -X or --request, POST is already inferred.
* Trying 192.168.56.101:80...
* Connected to 192.168.56.101 (192.168.56.101) port 80 (#0)
> POST /login HTTP/1.1
> Host: 192.168.56.101
> User-Agent: curl/8.7.1
> Content-Type: application/x-www-form-urlencoded
> Content-Length: 34
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< Content-Type: text/html; charset=utf-8
< Content-Length: 35
< Server: Werkzeug/3.0.3 Python/3.11.9
< Date: Mon, 18 Aug 2026 10:22:31 GMT
<
Login successful (HTTP)!
```

Fig-III: HTTP login request sent from the Kali machine to the target server.

- HTTP — Credentials Visible in Wireshark

Wireshark captures the HTTP POST request and displays the submitted form fields. The username student and password LabPass@2026 are visible in readable form, demonstrating that HTTP does not encrypt the login credentials.

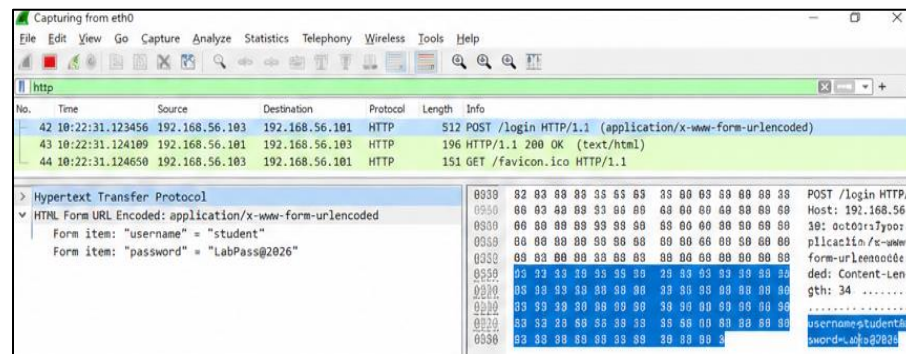


Fig-IV: Wireshark captures the HTTP POST request with the username and password visible in plaintext.

- HTTPS — Sending the Same Login

The same login credentials are submitted again, but this time through HTTPS. The -k option allows the connection to proceed despite the lab using a self-signed certificate.

```
# Sending the same login over HTTPS
(kali@kali)-[~]
$ curl -k -X POST https://192.168.56.101/login \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -d "username=student&password=LabPass@2026"
< HTTP/1.1 200 OK
< Date: Sun, 18 May 2026 11:36:25 GMT
< Server: Werkzeug/3.0.3 Python/3.11.9
< Content-Type: text/html; charset=utf-8
< Content-Length: 33
< Connection: close
<
< Login successful (HTTPS)

# Note:
# The -k flag is used to ignore the self-signed certificate in this lab.

(kali@kali)-[~]
$
```

Fig-V: The same login credentials submitted over HTTPS using a self-signed certificate.

- HTTPS — Encrypted Traffic in Wireshark

Wireshark captures the HTTPS session as TLS 1.3 traffic. Instead of displaying the username and password, the capture shows TLS handshake messages and encrypted Application Data, demonstrating that the credentials cannot be read directly from the packet capture.

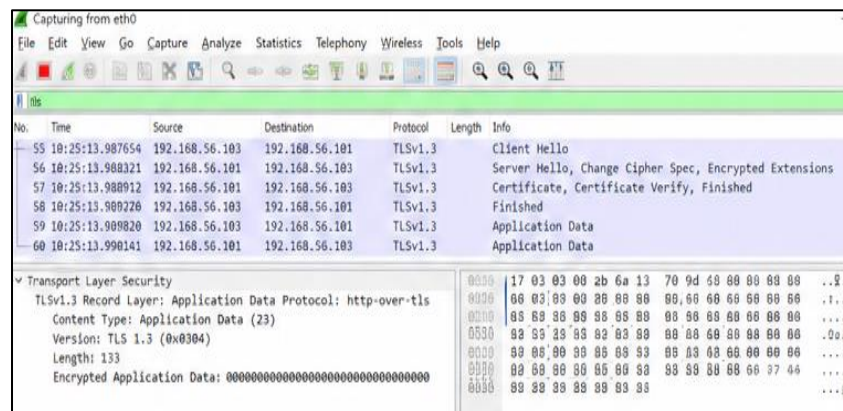


Fig-VI: Wireshark captures the HTTPS session as TLS 1.3 records with encrypted Application Data.

CONCLUSION

In conclusion, The Wireshark analysis clearly demonstrates the importance of cryptography in protecting sensitive information during transmission. Over HTTP, the dummy login credentials were transmitted in plaintext and could be directly observed in the captured traffic. In contrast, HTTPS used TLS encryption to protect the same credentials, making the captured application data unreadable to a network sniffer. Thus, the experiment shows that encryption is essential for maintaining the confidentiality and security of login credentials and other sensitive data while it travels across a network.